

# ***Documentación Técnica del Proyecto.***

*Sistema de Gestión de Clínica Médica Aurora.*

## ***Índice.***

|                                       |          |
|---------------------------------------|----------|
| <b>Índice.....</b>                    | <b>1</b> |
| <b>Introducción.....</b>              | <b>1</b> |
| <b>Objetivo general.....</b>          | <b>2</b> |
| <b>Objetivos específicos.....</b>     | <b>2</b> |
| <b>Arquitectura del sistema.....</b>  | <b>2</b> |
| Frontend.....                         | 2        |
| Backend (API REST).....               | 3        |
| Base de datos (MySQL).....            | 3        |
| Componentes principales.....          | 3        |
| Diagrama UML de las clases.....       | 3        |
| Diagrama de clases UML.....           | 3        |
| Significado del diagrama UML.....     | 6        |
| Backend (API REST).....               | 7        |
| Estructura de las carpetas.....       | 7        |
| Tecnologías y dependencias.....       | 8        |
| Modelos de los datos (ORM).....       | 8        |
| Relaciones según el diagrama UML..... | 8        |
| Frontend (Cliente Web).....           | 9        |
| Estructura de las carpetas.....       | 9        |
| Tecnologías usadas.....               | 9        |
| Diseño y estilos.....                 | 10       |
| Conexión con el backend.....          | 10       |
| Funcionalidad de los módulos.....     | 11       |
| Página principal.....                 | 11       |
| Dashboard.....                        | 11       |
| Módulo de padecimiento.....           | 11       |
| Módulo de especialidades.....         | 11       |
| Módulo de pacientes.....              | 11       |
| Módulo de médicos.....                | 11       |
| Módulo de citas médicas.....          | 12       |
| Módulo de reportes.....               | 12       |
| Conclusiones.....                     | 12       |
| Integrantes del grupo.....            | 12       |

## ***Introducción.***

*En el presente documento técnico se realiza la descripción de cómo fue desarrollado el Sistema de Gestión de Clínica Médica Aurora, también se menciona su diseño y diferentes implementaciones que se realizaron durante el desarrollo del programa. Este sistema consiste en una página web full stack, la cual fue desarrollada como un proyecto final del curso de Desarrollo de Software III. El propósito de este sistema, es poder optimizar y facilitar la administración de la información en una clínica, mediante el uso de una arquitectura robusta y concreta, implementando una interfaz gráfica muy intuitiva.*

*Asimismo, el tema del proyecto surge de la necesidad que muchas clínicas presentan en el día de hoy, como lo es digitalizar los diferentes procesos de gestión de información. Al realizarse este sistema, permite el registro, consultas, actualización y eliminación de los datos que presenta un paciente, e incluso el mismo doctor y clínica. También se lleva el registro del cuerpo médico, de los pacientes, padecimientos, citas y las diferentes especialidades. Este sistema consiste en la implementación de una arquitectura cliente-servidor, en donde el backend está desarrollado con FastAPI, usando el lenguaje de Python, por otro lado, el frontend fue desarrollado mediante el HTML, CSS y JavaScript vanilla, basándonos en un ámbito de tener un programa limpio, organizado, escalable y que sea fácil de llegar a actualizar en un futuro.*

*El propósito de esta documentación, es poder explicar la estructura, lógica, arquitectura que se implementa, de igual forma, se menciona las diferentes clases que fueron usadas, tanto en el backend como en el frontend. Detallando todos los componentes que fueron implementados por el backend, la estructura del frontend, las instrucciones para el buen funcionamiento y el modelo de los datos.*

### **Objetivo general.**

*Desarrollar un sistema de gestión de una clínica médica, el cual permita el manejo eficiente de los datos de los pacientes, citas, médicos, padecimientos y especialidades, facilitando el trabajo de todos los días.*

### **Objetivos específicos.**

- 1. Implementar un backend que sea robusto, mediante el uso de FastAPI, con los endpoints bien definidos para cada uno de los distintos módulos del sistema, logrando un CRUD completo y concreto.*
- 2. Diseñar una base de datos que sea relacional y normalizada en MySQL, siendo gestionada mediante el SQLAlchemy como ORM.*
- 3. Desarrollar un frontend que sea interactivo, llamativo y responsivo, el cual esté conectado al API, permitiendo que el usuario tenga una experiencia más profesional.*
- 4. Incorporar diferentes módulos de reportes, los cuales permitan una apreciación de las estadísticas claves del sistema.*
- 5. Garantizar la buena separación de las responsabilidades, como lo indica el MVC en el backend, y una estructura limpia y optimizada en el frontend, facilitando la actualización de futuros cambios.*

### **Arquitectura del sistema.**

*Para el desarrollo de este sistema, se tomó la decisión de trabajar con una arquitectura cliente-servidor, la cual se divide en tres diferentes capas:*

#### **Frontend.**

*El frontend se basa en el uso de HTML, CSS, JavaScript, Navegador Web y Live Server, todo esto permite:*

- 1. Una interfaz de usuario responsive.*
- 2. Un consumo de la API mediante el uso de Fetch con async y await.*
- 3. Una gestión dinámica de los diferentes módulos y estados.*

## Backend (API REST).

El backend fue basado en el uso de FastAPI, Python y Uvicorn, permitiendo:

1. Controladores (Routers).
2. Repositorios (Acceso a los datos).
3. Esquemas Pydantic (Validaciones).
4. Servicios (Logica del negocio).

## Base de datos (MySQL).

La base de datos fue basada en el uso de MySQL, gracias a ello se logró:

1. Tablas de medico, especialidad, paciente, padecimiento, cita\_medica.
2. Modelos ORM (SQLAlchemy).

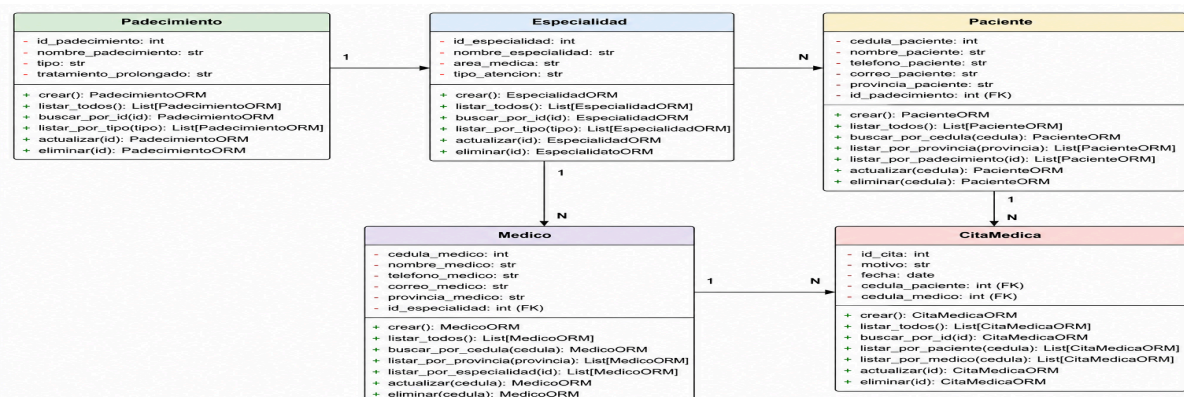
## Componentes principales.

| Capa           | Tecnología                 | Responsabilidad                                                      |
|----------------|----------------------------|----------------------------------------------------------------------|
| Frontend.      | HTML5, CSS3 y JavaScript.  | Consumo de API, interfaz de usuario y renderizado dinamico.          |
| Backend.       | Python, FastAPI y Uvicorn. | Validaciones, logica del negocio y exposición de los endpoints REST. |
| Base de datos. | MySQL y SQLAlchemy.        | Relaciones entre las entidades y persistencia de los datos,          |

## Diagrama UML de las clases.

El diagrama es uno de los componentes más importantes dentro de los proyectos, ya que indican la estructura de las entidades, así mismo los atributos y las relaciones que existen, a continuación se mostrará nuestro diagrama

## Diagrama de clases UML.



|                                                         |
|---------------------------------------------------------|
| <i>Padecimiento</i>                                     |
| - <i>id_padecimiento: int</i>                           |
| - <i>nombre_padecimiento: str</i>                       |
| - <i>tipo: str</i>                                      |
| - <i>tratamiento_prolongado: str</i>                    |
| + <i>crear (): PadecimientoORM</i>                      |
| + <i>listar_todos (): List [PadecimientoORM]</i>        |
| + <i>buscar_por_id (id): PadecimientoORM</i>            |
| + <i>listar_por_tipo (tipo): List [PadecimientoORM]</i> |
| + <i>actualizar (id): PadecimientoORM</i>               |
| + <i>eliminar (id): PadecimientoORM</i>                 |

|                                                         |
|---------------------------------------------------------|
| <i>Especialidad</i>                                     |
| - <i>id_especialidad: int</i>                           |
| - <i>nombre_especialidad: str</i>                       |
| - <i>area_medica: str</i>                               |
| - <i>tipo_atencion: str</i>                             |
| + <i>crear (): EspecialidadORM</i>                      |
| + <i>listar_todos (): List [EspecialidadORM]</i>        |
| + <i>buscar_por_id (id): EspecialidadORM</i>            |
| + <i>listar_por_tipo (tipo): List [EspecialidadORM]</i> |
| + <i>actualizar (id): EspecialidadORM</i>               |
| + <i>eliminar (id): EspecialidadORM</i>                 |

|                               |
|-------------------------------|
| <i>Paciente</i>               |
| - <i>cedula_paciente: int</i> |
| - <i>nombre_paciente: str</i> |

|                                                        |
|--------------------------------------------------------|
| - telefono_paciente: str                               |
| - correo_paciente: str                                 |
| - provincia_paciente: str                              |
| - id_padecimiento: int (FK)                            |
| + crear (): PacienteORM                                |
| + listar_todos (): List [PacienteORM]                  |
| + buscar_por_cedula (cedula): PacienteORM              |
| + listar_por_provincia (provincia): List [PacienteORM] |
| + listar_por_padecimiento (id): List [PacienteORM]     |
| + actualizar (cedula): PacienteORM                     |
| + eliminar (cedula): PacienteORM                       |

|                                                      |
|------------------------------------------------------|
| Medico                                               |
| - cedula_medico: int                                 |
| - nombre_medico: str                                 |
| - telefono_medico: str                               |
| - correo_medico: str                                 |
| - provincia_medico: str                              |
| - id_especialidad: int (FK)                          |
| + crear (): MedicoORM                                |
| + listar_todos (): List [MedicoORM]                  |
| + buscar_por_cedula (cedula): MedicoORM              |
| + listar_por_provincia (provincia): List [MedicoORM] |
| + listar_por_especialidad(id): List [MedicoORM]      |
| + actualizar (cedula): MedicoORM                     |
| + eliminar (cedula): MedicoORM                       |

|                                                                             |
|-----------------------------------------------------------------------------|
| <i>CitaMedica</i>                                                           |
| - <i>id_cita</i> : <i>int</i>                                               |
| - <i>motivo</i> : <i>str</i>                                                |
| - <i>fecha</i> : <i>date</i>                                                |
| - <i>cedula_paciente</i> : <i>int</i> (FK)                                  |
| - <i>cedula_medico</i> : <i>int</i> (FK)                                    |
| + <i>crear</i> (): <i>CitaMedicaORM</i>                                     |
| + <i>listar_todos</i> (): <i>List [CitaMedicaORM]</i>                       |
| + <i>buscar_por_id</i> ( <i>id</i> ): <i>CitaMedicaORM</i>                  |
| + <i>listar_por_paciente</i> ( <i>cedula</i> ): <i>List [CitaMedicaORM]</i> |
| + <i>listar_por_medico</i> ( <i>cedula</i> ): <i>List [CitaMedicaORM]</i>   |
| + <i>actualizar</i> ( <i>id</i> ): <i>CitaMedicaORM</i>                     |
| + <i>eliminar</i> ( <i>id</i> ): <i>CitaMedicaORM</i>                       |

| <i>Entidad</i>      | <i>Cardinalidad</i> | <i>Entidad</i>    |
|---------------------|---------------------|-------------------|
| <i>Padecimiento</i> | $1 \rightarrow N$   | <i>Paciente</i>   |
| <i>Especialidad</i> | $1 \rightarrow N$   | <i>Medico</i>     |
| <i>Paciente</i>     | $1 \rightarrow N$   | <i>CitaMedica</i> |
| <i>Medico</i>       | $1 \rightarrow N$   | <i>CitaMedica</i> |

### **Significado del diagrama UML.**

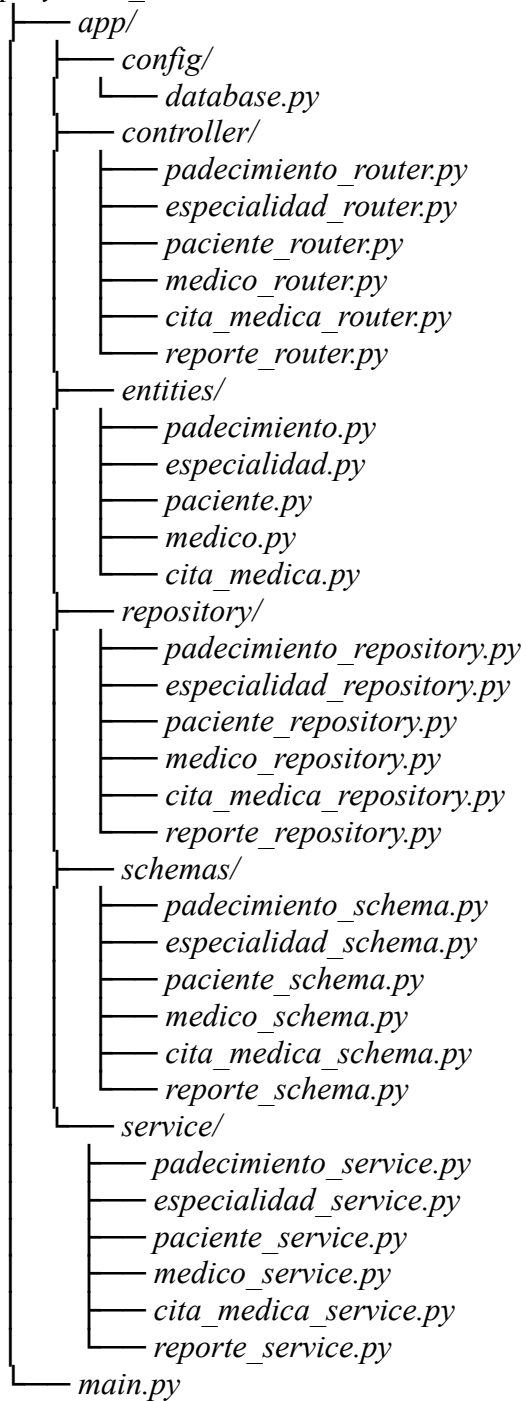
| <i>Símbolo</i> | <i>Significado</i>                |
|----------------|-----------------------------------|
| -              | <i>Atributo privado.</i>          |
| +              | <i>Atributo público.</i>          |
| <i>int</i>     | <i>Dato tipo entero.</i>          |
| <i>str</i>     | <i>Dato tipo cadena de texto.</i> |
| <i>date</i>    | <i>Dato tipo fecha.</i>           |

|              |                                      |
|--------------|--------------------------------------|
| <i>FK</i>    | <i>Clave foránea.</i>                |
| $\leftarrow$ | <i>Relación de las dependencias.</i> |
| <i>1 O N</i> | <i>Cardinalidad.</i>                 |

## **Backend (API REST).**

### **Estructura de las carpetas.**

proyecto2\_backend/



### ***Tecnologías y dependencias.***

| <i>Tecnologías</i>      | <i>Dependencias</i>                                       |
|-------------------------|-----------------------------------------------------------|
| <i>uvicorn</i>          | <i>Servidor ASGI para poder ejecutar el FastAPI.</i>      |
| <i>sqlalchemy</i>       | <i>ORM para la interacción con la base de datos.</i>      |
| <i>fastapi</i>          | <i>Framework web para lograr desarrollar la API REST.</i> |
| <i>pymysql</i>          | <i>Conector MySQL para Python.</i>                        |
| <i>pydantic</i>         | <i>Validaciones de los datos.</i>                         |
| <i>python-multipart</i> | <i>Manejo de los formularios.</i>                         |

### ***Modelos de los datos (ORM).***

*El presente sistema posee cinco entidades principales, en donde cada una de ellas va a ser representada por una clase ORM:*

| <i>Entidad</i>      | <i>Atributos clave</i>                                                                                                  |
|---------------------|-------------------------------------------------------------------------------------------------------------------------|
| <i>Padecimiento</i> | <i>id_padecimiento, nombre_padecimiento, tipo y tratamiento_prolongado.</i>                                             |
| <i>Especialidad</i> | <i>id_especialidad, nombre_especialidad, area_medica y tipo_atencion.</i>                                               |
| <i>Paciente</i>     | <i>cedula_paciente, nombre_paciente, telefono_paciente, correo_paciente, provincia_paciente y id_padecimiento (FK).</i> |
| <i>Medico</i>       | <i>cedula_medico, nombre_medico, telefono_medico, correo_medico, provincia_medico, id_especialidad (FK).</i>            |
| <i>CitaMedica</i>   | <i>id_cita, motivo, fecha, cedula_paciente (FK), cedula_medico (FK).</i>                                                |

### ***Relaciones según el diagrama UML.***

- 1. Un Paciente tiene un Padecimiento.*
- 2. Un Medico tiene una Especialidad.*
- 3. Una CitaMedica tiene un Paciente y un Medico.*

*Estas tres relaciones que están presentes, son de muchos a uno.*

## **Frontend (Cliente Web).**

### **Estructura de las carpetas.**

```
proyecto2_frontend/
├── index.html
├── dashboard.html
├── css/
│   ├── dashboard.css
│   ├── estilos.css
│   └── index.css
├── html/
│   ├── module_padecimiento.html
│   ├── module_especialidad.html
│   ├── module_paciente.html
│   ├── module_medico.html
│   ├── module_citas.html
│   └── module_reportes.html
├── js/
│   ├── index.js
│   ├── api.js
│   ├── app.js
│   ├── dashboard.js
│   └── modules/
│       ├── padecimiento.js
│       ├── especialidad.js
│       ├── paciente.js
│       ├── medico.js
│       ├── citas.js
│       └── reportes.js
└── img/
    └── clinica3.jpg
```

### **Tecnologías usadas.**

| <i>Tecnología</i>  | <i>Propósito</i>                                               |
|--------------------|----------------------------------------------------------------|
| <i>HTML5</i>       | <i>Estructura semántica de las páginas.</i>                    |
| <i>CSS3</i>        | <i>Estilos , diseños y paletas de colores.</i>                 |
| <i>JavaScript</i>  | <i>Consumo de API mediante el DOM y la logica del negocio.</i> |
| <i>Fetch API</i>   | <i>Conexión con el backend.</i>                                |
| <i>Live Server</i> | <i>Servidor de desarrollo para el frontend.</i>                |

### **Diseño y estilos.**

Toda página se caracteriza por sus estilos y colores llamativos, en este sistema se tomó la decisión de trabajar con una estructura sencilla, pero bien organizada, con colores claros para hacer la pagina más comoda y moderna, para lograr todo eso, se hizo uso de:

1. #F5EDE3 para los fondos generales (beige claro).
2. #FFFFFF para las tarjetas y contenedores (blanco).
3. #4A3226 para los textos y títulos principales (café oscuro).
4. #8B6B4D para los detalles (café medio pálido).
5. #D4C4B0 para los separadores y bordes (beige).
6. #5C3D2E para el footer y los botones (café intermedio).

### **Conexión con el backend.**

En este tipo de trabajos, es muy importante la conexión y comunicación entre el frontend y el backend, gracias a ello, esta comunicación se puede dar por medio de la función de petición API () , la cual fue definida en la clase de JavaScript de api.

```
-La clase se encarga de la conexión con la API
-*/
const API_BASE_URL = 'http://127.0.0.1:8000';

async function peticionAPI(url, metodo, datos){
  const opciones = {
    method: metodo,
    headers: {
      'Content-Type': 'application/json'
    }/*fin del headers*/
  };/*fin de opciones*/
  if (datos){
    opciones.body = JSON.stringify(datos);
  }/*fin del if*/

  try{
    const respuesta = await fetch(url, opciones);
    if (!respuesta.ok){
      const errorData = await respuesta.json();
      const error = {
        status: respuesta.status,
        mensaje: errorData.detail || respuesta.statusText
      };/*fin de error*/
      throw error;
    }/*fin del if*/

    const datosRespuesta = await respuesta.json();
    return datosRespuesta;
  }catch (error){
    if (error.status === undefined){
      const errorConexion = {
        status: 0,
```

```
    mensaje: 'Error de conexion con el servidor'
};/*fin de errorConexion*/
throw errorConexion;
}/*fin del if*/
throw error;
}/*fin del try*/
}/*fin de peticionAPI*/
```

## **Funcionalidad de los módulos.**

### **Página principal.**

*Al momento de que se ingresa a index.html, se puede apreciar las seis tarjetas que se encargan de representar los distintos módulos, y también se aprecia el nombre de la clínica, y el footer.*

### **Dashboard.**

*Contiene un menú lateral al lado izquierdo de la pantalla, en donde salen los seis diferentes módulos que están disponibles, cuando se toca el botón del módulo que se desea, se carga otra interfaz en donde ya salen los botones, tablas, menuBar y espacios para los datos.*

### **Módulo de padecimiento.**

*El módulo presenta varias funciones, como registrar, buscar por ID, listar, buscar por tipo, eliminar y actualizar. Es importante resaltar que hay un menuBar que solo va a aparecer si se toca que si necesita tratamiento prolongado, ese menuBar es para determinar la durabilidad. Cada una de estas funciones representan a un endpoint, en donde cada uno debe de cumplir con una serie de requisitos.*

### **Módulo de especialidades.**

*Este módulo es muy similar al módulo de padecimiento, posee las mismas funciones, lo único que varía es el tipo de datos que solicita, el tipo de búsqueda que realiza y los datos que retorna al momento de buscar y de listar.*

### **Módulo de pacientes.**

*Este módulo es muy similar a los primeros módulos de los que se habló, posee las mismas funciones, lo único que varía es el tipo de datos que solicita, el tipo de búsqueda que realiza y los datos que retorna al momento de buscar y de listar.*

### **Módulo de médicos.**

*Este módulo es muy similar a los primeros módulos de los que se habló, posee las mismas funciones, lo único que varía es el tipo de datos que solicita, el tipo de búsqueda que realiza y los datos que retorna al momento de buscar y de listar.*

### ***Módulo de citas médicas.***

*Este módulo es muy similar a los primeros módulos de los que se habló, posee las mismas funciones, lo único que varía es el tipo de datos que solicita, el tipo de búsqueda que realiza y los datos que retorna al momento de buscar y de listar.*

### ***Módulo de reportes.***

*A diferencia de los demás módulos, este módulo solo posee tres botones, los cuales representan los tres tipos de reportes que existen en el sistema, solo se escoge el botón, y posteriormente saldrá la información que se desea ver.*

### ***Conclusiones.***

*Con el desarrollo del presente proyecto, creando un sistema de gestión de una clínica médica, logramos entender como el backend y el frontend trabajan de forma mutua, y de qué tan importante es la conexión entre ellos, también se logra fomentar el conocimiento de los diferentes elementos y componentes para lograr crear una página web, aprendiendo en que se usa cada una de ellas, desde el backend, frontend hasta el analista de datos. Durante este proceso se logró implementar una estructura sólida que se divide en tres capas diferentes, se llevó a cabo la creación de un CRUD completo, de igual forma que se realizó una interfaz creativa y intuitiva, posteriormente se realizaron tres tipos de reportes, a cada uno de los modelos se les implementó validaciones y seguridad. Lo más relevante de este proceso fue aprender como un proyecto que puede volver más fácil si se realiza una separación de tareas de la forma correcta y como hacer las cargas dinámicas de los módulos que se desean trabajar.*

### ***Integrantes del grupo.***

| <i>Integrante</i>    | <i>Rol</i>                 |
|----------------------|----------------------------|
| <i>David Chang</i>   | <i>Backend y Frontend.</i> |
| <i>Enzo Bejarano</i> | <i>Backend y Frontend.</i> |
| <i>Windell Urroz</i> | <i>Backend y Frontend.</i> |